

CoNarrative Codex CLI Implementation Packet

English diagrams • Plus-friendly Codex CLI setup • agent topology • model and reasoning guidance

Purpose: give Codex a practical, quota-aware implementation handoff for building the CoNarrative prototype.

Architecture baseline comes from the existing CoNarrative packet and presentation: Story Bible + State Tracker + Dual Critic + Revision.

1. Plus-plan operating strategy

OpenAI says Codex usage limits depend on your plan, task size, codebase size, and whether work runs locally or in the cloud. The exact number of usable Codex messages is therefore variable rather than a single fixed count. The official guidance is to monitor the Codex usage dashboard and the `/status`` command during active CLI sessions.

Because you are on ChatGPT Plus, the safest default is to optimize for local interactive work, keep sessions short, and avoid parallel subagents unless the task is clearly worth the extra token cost.

- Default to gpt-5.4-mini for everyday local work. OpenAI says Plus includes GPT-5.4-mini with up to 3.3x higher usage limits for local messages.
- Use gpt-5.4 when integrating multi-file changes, locking schemas, or resolving blockers.
- Use gpt-5.3-codex only for the hardest long-horizon coding tasks that still fail on gpt-5.4.
- Keep reasoning at medium by default. OpenAI recommends medium as the all-around interactive coding setting; reserve high or xhigh for hard blockers.
- Do not spawn subagents by default. OpenAI notes that each subagent performs its own model and tool work, so subagent workflows consume more tokens than comparable single-agent runs.

Scenario	Model	Reasoning	Why
Everyday coding / iteration	gpt-5.4-mini	medium	Best Plus-friendly default; higher local usage headroom.
Integration / difficult multi-file edits	gpt-5.4	medium	Stronger coding intelligence without immediately jumping to costly settings.
Schema, storage, and architecture blockers	gpt-5.4	high	Use only when the task is genuinely hard.
Long-horizon stubborn coding task	gpt-5.3-codex	high	Fallback escalation path for agentic coding jobs.
Emergency last resort	gpt-5.3-codex	xhigh	Use sparingly because it is slower and heavier.

2. Architecture diagrams in English

The figures below replace the previous broken Korean-rendered diagrams. They are ready to hand to a coding agent and match the current CoNarrative runtime design.

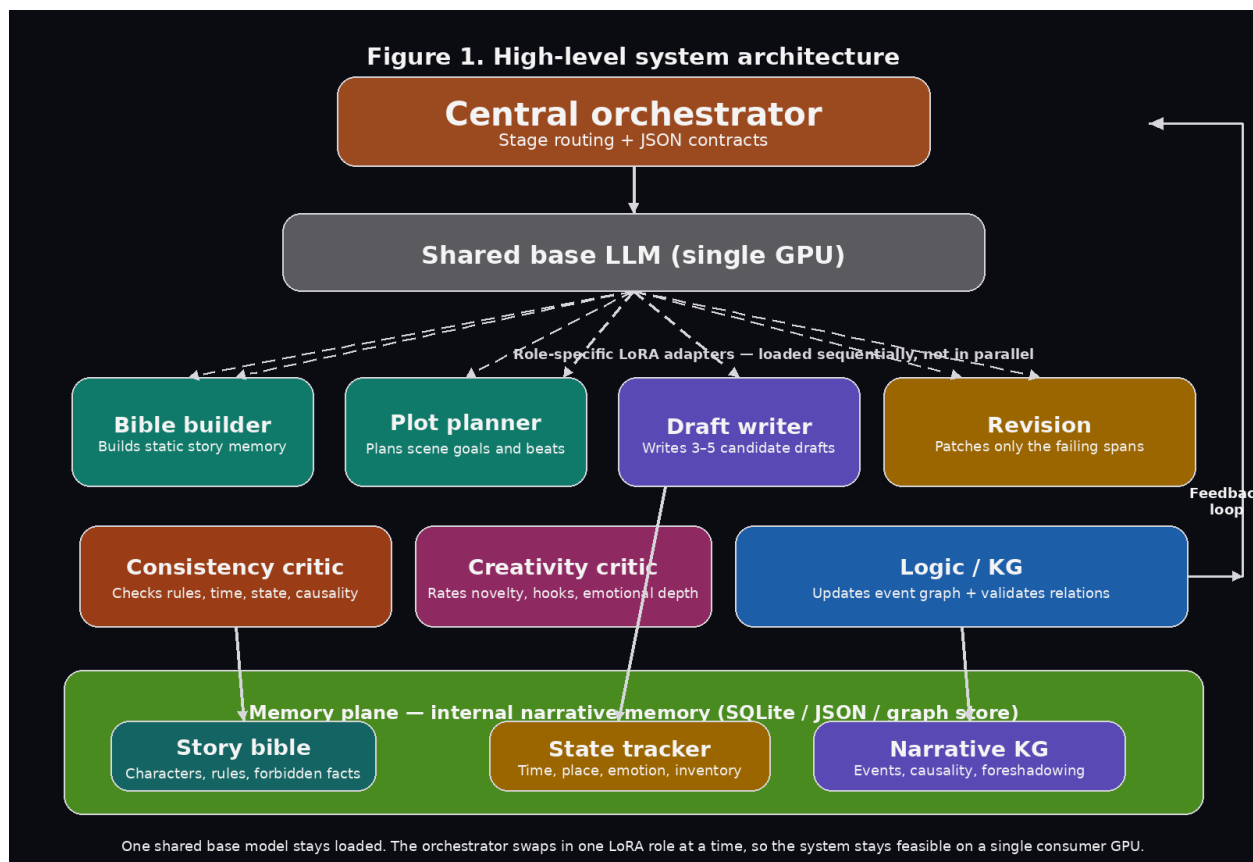


Figure 1. High-level system architecture

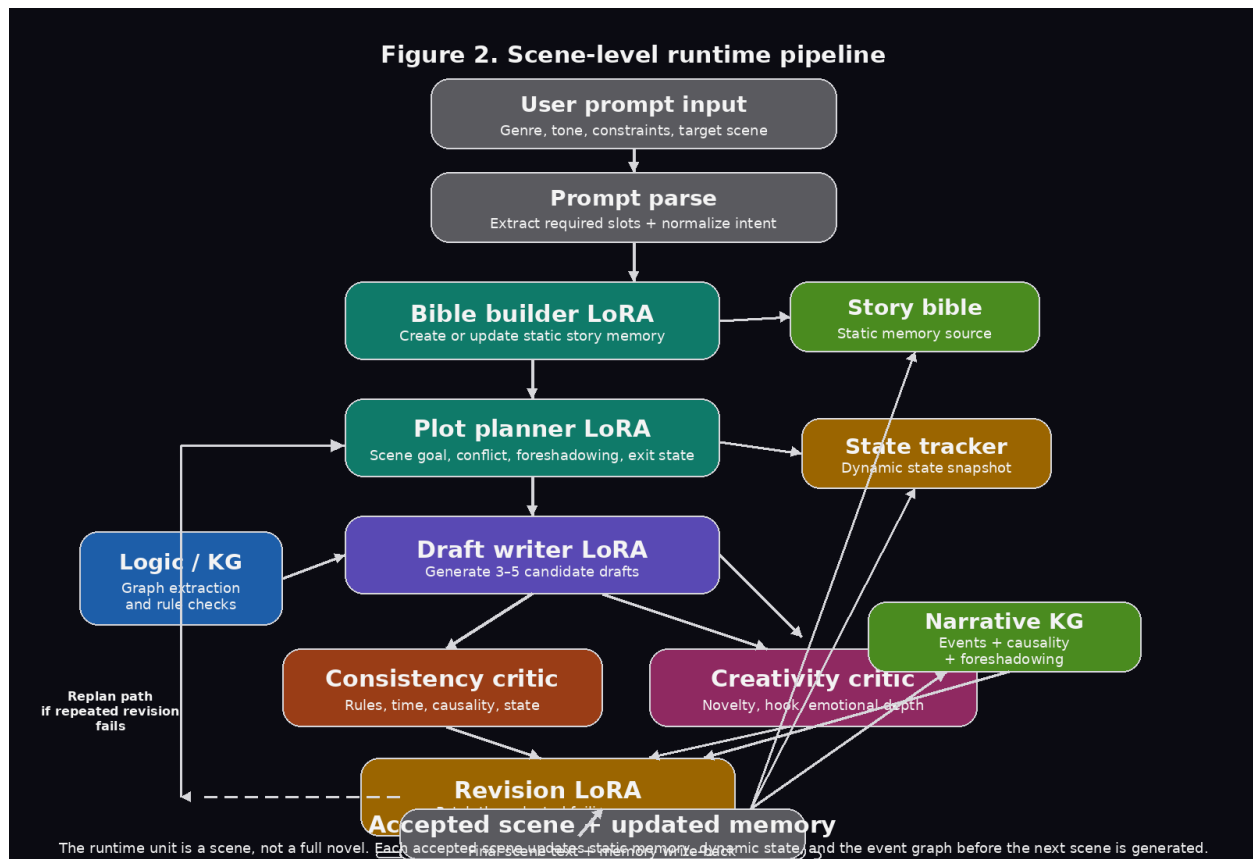


Figure 2. Scene-level runtime pipeline
CoNarrative Codex packet

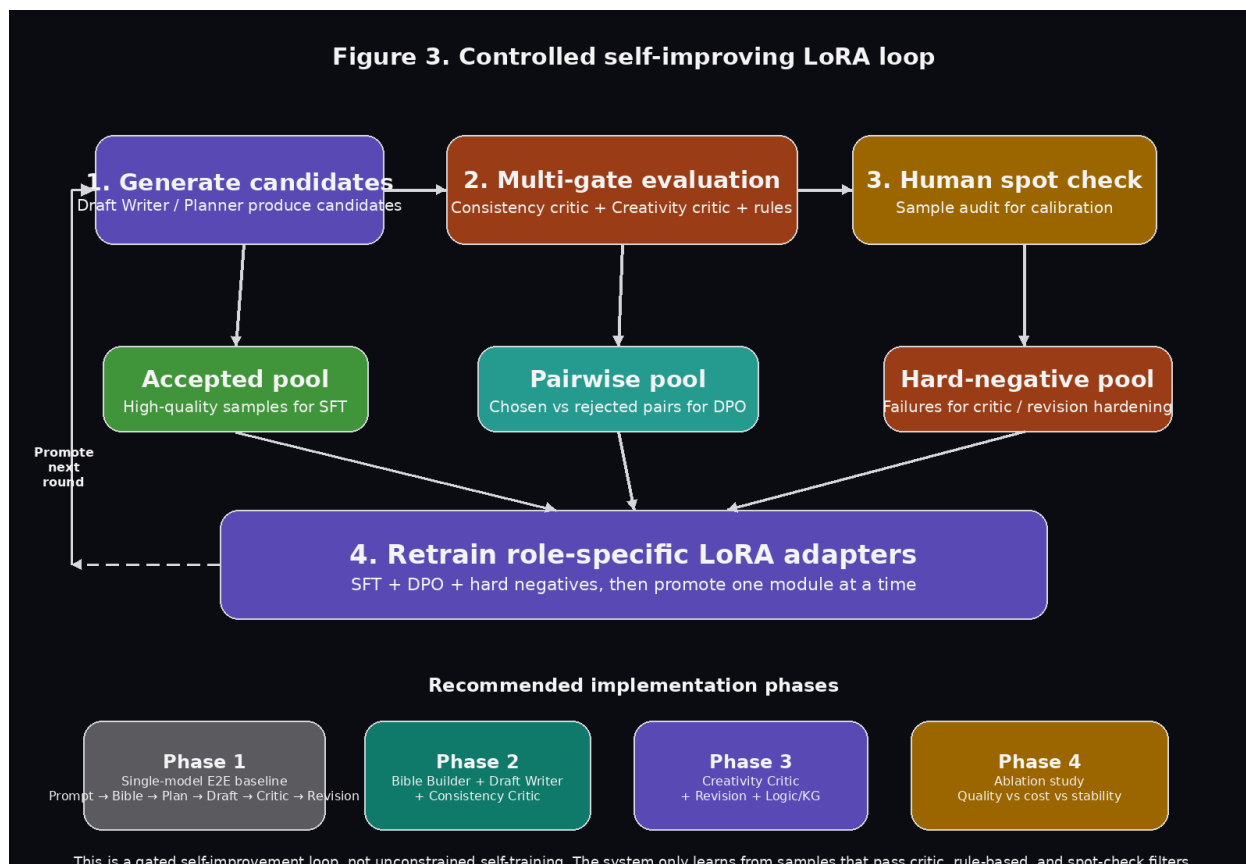


Figure 3. Controlled self-improving LoRA loop

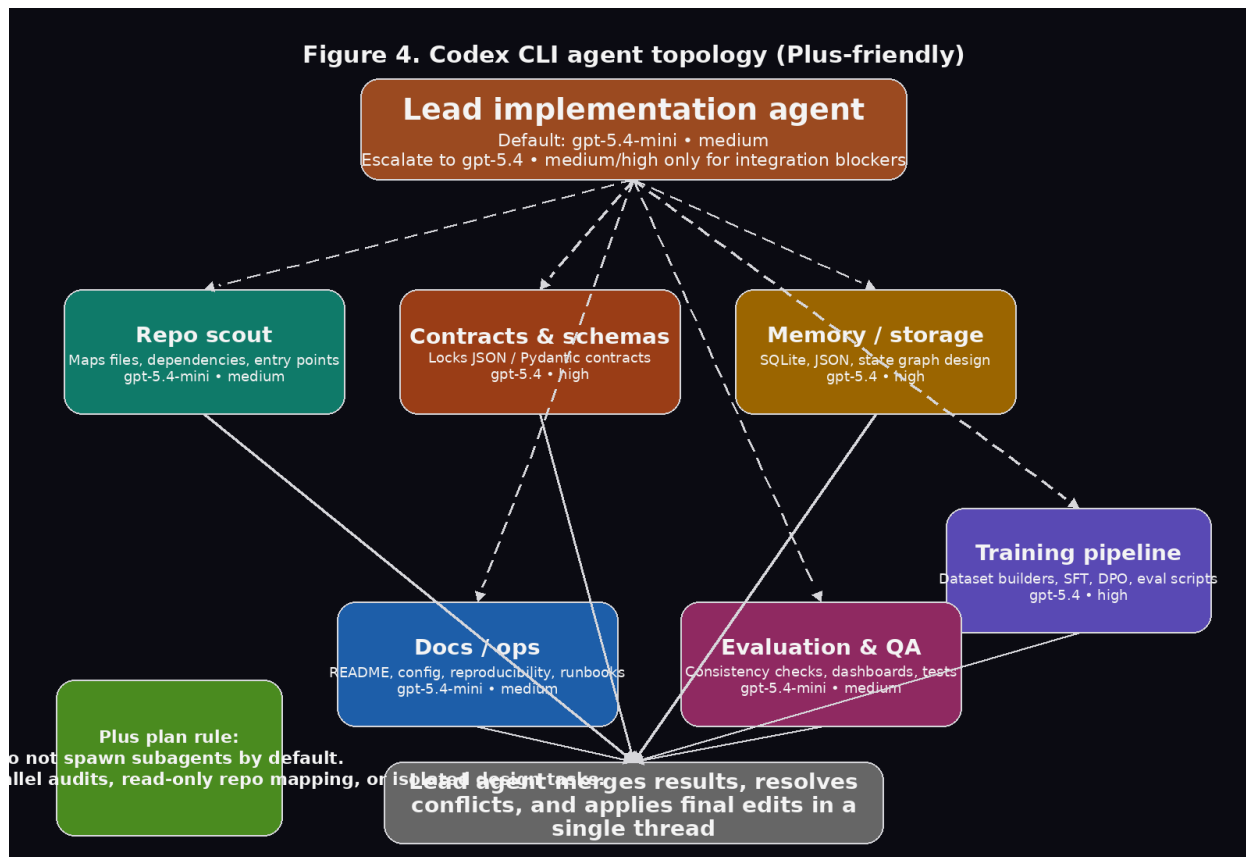


Figure 4. Codex CLI agent topology for ChatGPT Plus
CoNarrative Codex packet

3. Codex instruction strategy

Codex reads AGENTS.md before doing work, and OpenAI recommends using it for consistent project guidance. Custom subagents live in ``.codex/agents/*.toml``, each with a name, description, developer instructions, and optional model configuration. Profiles and defaults live in ``.config.toml``.

For this project, the best pattern is one lead agent that owns final edits, plus a small number of read-only specialist subagents used only when necessary. This keeps the main thread coherent and avoids wasteful parallel token burn.

- Lead agent owns planning, merges results, resolves conflicts, and applies the final code edits.
- Repo scout maps the codebase and returns file targets before implementation begins.
- Contracts & schemas agent locks JSON / Pydantic contracts so module boundaries stay stable.
- Memory / storage agent designs Story Bible, State Tracker, and Narrative KG persistence.
- Training pipeline agent designs dataset builders, SFT, DPO, and evaluation scripts.
- Evaluation & QA agent audits consistency metrics, tests, and dashboards.
- Docs / ops agent keeps README, runbooks, config examples, and reproducibility notes aligned with reality.

4. Ready-to-use AGENTS.md

Use the following repository-level AGENTS.md to make Codex follow the architecture instead of drifting into random rewrites.

```
# CoNarrative Codex instructions

## Mission
Build a working CoNarrative prototype for long-form story generation with explicit
memory, planning, dual critics, revision, and a controlled self-improving LoRA pipeline.

## Operating rules
- Keep the repository runnable at every meaningful checkpoint.
- Prefer the smallest change that preserves the architecture.
- Treat JSON or Pydantic schemas as API contracts. Do not change them casually.
- Separate static memory, dynamic state, and event-graph storage.
- Keep all model-facing prompts and schemas versioned in the repo.
- Before adding dependencies, explain why they are needed and prefer lightweight
defaults.
- Run focused validation after each code change. Do not claim success without command
output.
- Preserve reproducibility: document commands, config, seeds, and expected outputs.

## Implementation priorities
1. End-to-end baseline chain first.
2. Schema stability second.
3. Memory correctness third.
4. Critic and revision loop fourth.
5. LoRA training and evaluation pipeline after the baseline is stable.

## Architectural invariants
- The runtime unit is a scene, not a full novel.
- One shared base model stays loaded; role-specific LoRAs are swapped sequentially.
- Story Bible stores static facts.
- State Tracker stores per-scene dynamic state.
- Narrative KG stores events, causality, and foreshadowing edges.
- Revision patches failing spans; repeated failure triggers replanning.
- Self-improvement is gated: accepted, pairwise, and hard-negative pools are stored
```

separately.

Coding style

- Use explicit typing where practical.
- Prefer small modules with narrow responsibilities.
- Put orchestration policy in ``orchestrator.py``, not spread across model wrappers.
- Put schemas in one place.
- Log enough context to debug failures, but do not dump excessive prompt text into logs by default.

Validation checklist

- Does the baseline chain run from prompt parse to accepted scene?
- Does memory update correctly between adjacent scenes?
- Do critic outputs include machine-readable issue types and evidence spans?
- Is every generated artifact saved to a predictable path?
- Are training datasets accumulated into separate pools?

Subagent policy

- Do not spawn subagents by default on ChatGPT Plus.
 - Use subagents only for short read-only scouting, contract review, storage design, QA audits, or docs work.
 - Let the lead agent merge results and make final code edits in one thread.
-

5. Ready-to-use Codex config.toml

This config is intentionally conservative for ChatGPT Plus. It starts with gpt-5.4-mini at medium reasoning, sets a small subagent budget, and provides escalation profiles when needed.

```
model = "gpt-5.4-mini"
model_provider = "openai"
model_reasoning_effort = "medium"
model_verbosity = "medium"
approval_policy = "on-request"
personality = "pragmatic"
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536
```

```
[agents]
max_threads = 3
max_depth = 1
job_max_runtime_seconds = 900
```

```
[profiles.quick]
model = "gpt-5.4-mini"
model_reasoning_effort = "medium"
approval_policy = "on-request"
```

```
[profiles.integration]
model = "gpt-5.4"
model_reasoning_effort = "medium"
approval_policy = "on-request"
```

```
[profiles.deep]
model = "gpt-5.4"
model_reasoning_effort = "high"
approval_policy = "on-request"
```

```
[profiles.codex_hard]
model = "gpt-5.3-codex"
model_reasoning_effort = "high"
approval_policy = "on-request"
```

6. Recommended custom agents

OpenAI documents three built-in agents—default, worker, and explorer—and supports custom agent TOML files with per-agent model and reasoning settings. For CoNarrative, the following project-scoped custom agents are recommended.

Agent	Model	Reasoning	Sandbox	Primary job
repo_scout	gpt-5.4-mini	medium	read-only	Map code paths, files, dependencies, and entry points.
contracts_schema	gpt-5.4	high	read-only	Define JSON/Pydantic contracts and migration rules.
memory_storage	gpt-5.4	high	read-only	Design Story Bible, State Tracker, and KG persistence.
training_pipeline	gpt-5.4	high	read-only	Design dataset builders, SFT, DPO, and eval flow.
eval_qa	gpt-5.4-mini	medium	read-only	Audit metrics, tests, regressions, and QA coverage.
docs_ops	gpt-5.4-mini	medium	read-only	README, runbooks, reproducibility, and config docs.

repo_scout.toml

```
name = "repo_scout"
description = "Read-only codebase explorer for mapping files, entry points, and dependencies before implementation."
model = "gpt-5.4-mini"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"
developer_instructions = """
Stay in exploration mode.
Map the execution path, identify key files and symbols, and return a concise plan with file references.
Do not propose broad rewrites. Do not edit code.
Prefer fast search and targeted file reads over broad scans.
"""
```

contracts_schema.toml

```
name = "contracts_schema"
description = "Schema specialist for JSON and Pydantic contracts across the CoNarrative pipeline."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = ""
Define or review schemas for prompt parse, planning, drafting, critics, revision, and memory updates.
Favor stable fields, explicit enums, and backward-compatible changes.
Return exact contract proposals and migration notes.
Do not edit code unless explicitly asked by the parent agent.
""
```

memory_storage.toml

```
name = "memory_storage"
description = "Storage design specialist for Story Bible, State Tracker, and Narrative KG."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = ""
Design SQLite, JSON, and graph-store structures for internal narrative memory.
Keep static memory, dynamic state, and event graph clearly separated.
Emphasize debuggability, deterministic updates, and scene-to-scene consistency.
Return table layouts, indexes, and update rules.
""
```

training_pipeline.toml

```
name = "training_pipeline"
description = "Training specialist for dataset builders, SFT, DPO, hard-negative mining, and evaluation scripts."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = ""
Design the controlled self-improvement loop.
Separate accepted, pairwise, prompt-only, and hard-negative pools.
Prefer reproducible scripts, explicit file formats, and phase-by-phase rollout.
Return command sequences, config files, and expected outputs.
""
```

eval_qa.toml

```
name = "eval_qa"
description = "Evaluation and QA specialist for consistency checks, dashboards, and test strategy."
model = "gpt-5.4-mini"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"
```

```
developer_instructions = """
Audit correctness and measurement.
Focus on setting conflicts, timeline errors, state drift, unresolved foreshadowing, and
regression coverage.
Return concrete tests, metrics, and failure cases with minimal commentary.
Do not edit application code.
"""
```

docs_ops.toml

```
name = "docs_ops"
description = "Docs and ops specialist for README, runbooks, config examples, and
reproducibility notes."
model = "gpt-5.4-mini"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"
developer_instructions = """
Write concise developer-facing documentation.
Prefer exact commands, directory layouts, environment notes, and troubleshooting steps.
Keep docs synchronized with actual file names and scripts.
Do not invent commands that were not verified.
"""
```

7. Recommended prompting patterns for Codex

These prompts are designed to reduce wasted turns. They tell Codex exactly when to explore, when to edit, and when to stop.

7.1 Lead implementation prompt

You are the lead implementation agent for CoNarrative.

Goal: implement the smallest vertical slice that makes the baseline runtime chain work end to end.

Rules:

- Read AGENTS.md and obey the architecture invariants.
 - First map the exact files you will touch.
 - Keep JSON / Pydantic contracts stable unless a schema change is unavoidable.
 - Separate static memory, dynamic state, and event graph logic.
 - After edits, run the smallest relevant validation commands and report exact results.
 - If you are uncertain about repo structure, ask repo_scout for a read-only map instead of guessing.
 - Do not spawn more than one subagent unless explicitly asked.
-

7.2 Repo scouting prompt

Spawn repo_scout in read-only mode. Ask it to map the files, entry points, existing schemas, and likely test commands for the runtime baseline. Wait for its answer, then summarize the file plan before you edit anything.

7.3 Schema locking prompt

Spawn `contracts_schema` in read-only mode. Ask it to propose stable contracts for `prompt_parse_output`, `plan_output`, `draft_output`, `critic_output`, `revision_output`, and `memory_update`. Require backward-compatible field names and explicit enums where possible.

7.4 Evaluation prompt

Spawn `eval_qa` in read-only mode. Ask it to list the minimum tests and metrics needed to catch setting conflicts, timeline errors, state drift, and unresolved foreshadowing. Require concrete file targets and command suggestions.

8. Concrete implementation order for Codex

Phase	What Codex should build first	Suggested profile
1	Schemas, orchestrator stub, memory stores, and a fake or prompt-only baseline chain.	quick
2	Prompt parse -> Bible -> Plan -> Draft -> Critic -> Revision path with scene-level memory update.	integration
3	Consistency checks, revision logs, and metrics collection.	integration
4	Dataset builders and controlled self-improvement pools: accepted, pairwise, hard-negative.	deep
5	LoRA training scripts and evaluation automation.	deep or codex_hard only if needed

For Plus, keep each Codex run narrow. A good working rhythm is: one repo mapping turn, one implementation turn, one validation turn, then stop and reassess ``/status``. Avoid marathon sessions until the baseline chain is stable.

9. Research basis

The architecture and Codex recommendations below are grounded in current official OpenAI documentation and recent story-generation research.

Source	Why it matters
Gemma 4 model card and tuning docs	Supports the shared-base-model plus role-specific adaptation idea and small-data LoRA tuning.
Codex CLI docs, config docs, AGENTS.md guide, and subagent docs	Define how to structure instructions, profiles, models, reasoning effort, and custom agents.
Codex prompting guide	Recommends medium reasoning as the default interactive coding setting; reserve high/xhigh for harder work.
Codex pricing and ChatGPT plan docs	Explain that Plus includes GPT-5.4, GPT-5.3-Codex, GPT-5.4-mini, and variable usage limits.
DOME and StoryWriter	Support planning + memory + multi-agent separation for long-form story generation.
LongStoryEval and Lost in Stories	Show that long story consistency needs dedicated

	evaluation axes.
Self-Refine, DPO, Online DPO, TRL trainers	Support the controlled self-improving LoRA loop rather than naive self-training.

10. References

1. OpenAI Developers. Codex CLI. <https://developers.openai.com/codex/cli/>
2. OpenAI Developers. Codex models. <https://developers.openai.com/codex/models/>
3. OpenAI Developers. Config basics / sample / advanced config.
<https://developers.openai.com/codex/config-basic/> ; <https://developers.openai.com/codex/config-sample/> ;
<https://developers.openai.com/codex/config-advanced/>
4. OpenAI Developers. Subagents. <https://developers.openai.com/codex/subagents/>
5. OpenAI Developers. Custom instructions with AGENTS.md.
<https://developers.openai.com/codex/guides/agents-md/>
6. OpenAI Help. Using Codex with your ChatGPT plan. <https://help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan>
7. OpenAI Developers. Codex pricing. <https://developers.openai.com/codex/pricing/>
8. OpenAI Cookbook. Codex Prompting Guide.
https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide/
9. OpenAI API docs. Reasoning models / GPT-5.4 / GPT-5.3-Codex.
10. Wang et al. DOME. NAACL 2025.
11. Xia et al. StoryWriter. 2025.
12. Yang & Jin. LongStoryEval. ACL 2025.
13. Li et al. Lost in Stories. 2026.
14. Madaan et al. Self-Refine. 2023.
15. Rafailov et al. DPO. 2023.
16. Hugging Face TRL. SFT Trainer / DPO Trainer / Online DPO Trainer docs.